HDOC DAY-60

Write a C program that reads two strings, s and t. Assume that both strings contain only lowercase characters. Pass the strings to a function named check_anagram() using their addresses. Use pointers to pass the strings between the calling function and check_anagram(). The function check_anagram() must:

- Determine whether s and t contain the same characters with the same frequencies.
- Print "Anagram" if the two strings are valid anagrams.
- Print "Not Anagram" if the two strings are not valid anagrams.
- Treat the order of characters as irrelevant.
- Consider strings of different lengths as not anagrams.
- Use a character-frequency-counting approach to solve the problem.

Example 1:

String s: listen
String t: silent

Output: Anagram

Example 2:

String s: hello
String t: world

Output: Not Anagram

Example 3:

String s: anagram
String t: nagaram

Output: Anagram

Example 4:

String s: rat
String t: car

Output: Not Anagram

Algorithm:

1. **Start.**
2. Read two strings s and t.
3. Pass the addresses of s and t to the function check_anagram().
4. Create an integer array freq[26] and initialize all its elements to 0.
5. Compare the lengths of s and t.
    - If the lengths are different, print **"Not Anagram"** and return.
6. Traverse string s using a pointer:
    - For each character, increment its corresponding frequency in freq[].
7. Traverse string t using a pointer:
    - For each character, decrement its corresponding frequency in freq[].
8. Traverse freq[] from index 0 to 25.
    - If any element is not 0, print **"Not Anagram"** and return.
9. If all elements are 0, print **"Anagram"**.
10. **Stop.**

- C Program:
- #include <stdio.h>
- #include <string.h>
-
- void check_anagram(char *s, char *t)
- {
- int freq[26] = {0};
-
- // Different lengths cannot be anagrams
- if (strlen(s) != strlen(t))
- {
- printf("Not Anagram");
- return;
- }

```
    // Count characters in s and subtract characters in t
    while (*s != '\0')
    {
        freq[*s - 'a']++;
        s++;
    }

    while (*t != '\0')
    {
        freq[*t - 'a']--;
        t++;
    }

    // Check whether all frequencies are zero
    for (int i = 0; i < 26; i++)
    {
        if (freq[i] != 0)
        {
            printf("Not Anagram");
            return;
        }
    }

    printf("Anagram");
}

int main()
{
    char s[100], t[100];

    printf("Enter string s: ");
    scanf("%99s", s);

    printf("Enter string t: ");
    scanf("%99s", t);

    check_anagram(s, t);

```

- return 0;
}

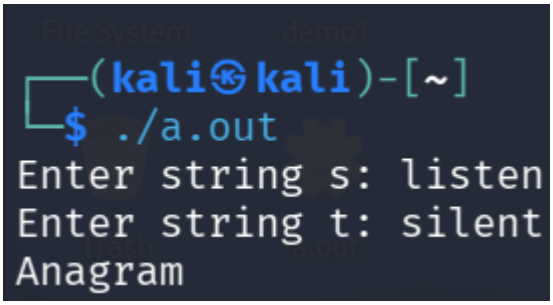
Test Case Table:c

Test Case	String (s)	String (t)	Expected Output	Remark
1	listen	silent	Anagram	Same characters with same frequencies
2	hello	world	Not Anagram	Characters are different
3	anagram	anagram	Anagram	Same characters in different order
4	rat	car	Not Anagram	Characters are different
5	abc	abcd	Not Anagram	Different string lengths
6	aabb	abab	Anagram	Same character frequencies
7	aabb	abbb	Not Anagram	Character frequencies are different
8	abc	abc	Anagram	Identical strings are also valid anagrams

Compilation and Execution:

The following program has been compiled and executed successfully.

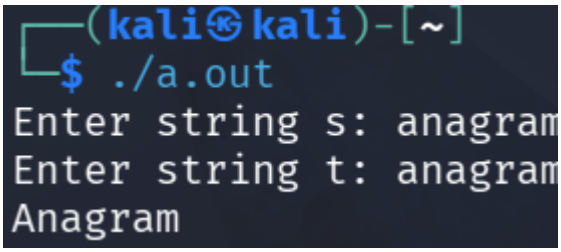
Test Case 1:



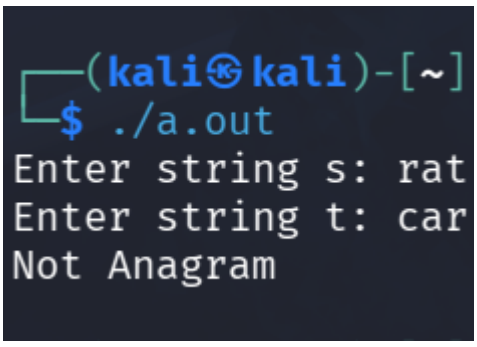
Test Case 2:



Test Case 3:



Test Case 4:



Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter string s: abc
Enter string t: abcd
Not Anagram
```

Test Case 6:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter string s: aabb
Enter string t: abab
Anagram
```

Test Case 7:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter string s: aabb
Enter string t: abbb
Not Anagram
```

Test Case 8:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter string s: abc
Enter string t: abc
Anagram
```